



UNIVERSIDAD PRIVADA DEL VALLE
FACULTAD DE INFORMÁTICA Y
ELECTRÓNICA
INGENIERÍA DE SISTEMAS INFORMÁTICOS

Evaluación

SISTEMA DE AEROLINEA RECLAMOS
MANUAL TECNICO PROGRAMACION WEB

Paralelo: “A”

Estudiantes: Danilo Arinez Sánchez
Cuba Condori Ludwing Leonel
Paucara Mamani Oliver Yassir

Docente: Ing. Henry Miranda Ordoñez

La Paz 10 de Junio de 2026

Gestión I – 2026

Contenido

1. Introducción	1
2. Objetivo del sistema	1
3. Arquitectura del proyecto	2
4. Tecnologías utilizadas	2
Backend	2
Frontend	2
Herramientas de desarrollo	3
5. Requisitos para ejecutar el proyecto	3
6. Estructura del backend	3
7. Estructura del frontend	4
8. Configuración del archivo .env	4
9. Instalación del backend	5
10. Instalación del frontend	5
11. Modelo relacional normalizado	6
Relaciones principales	6
12. Normalización y reducción de redundancia	8
13. Modelos principales	9
14. Controladores API	10
15. Autenticación	11
16. Usuarios de prueba	11
17. Rutas principales de la API	11
Rutas de autenticación	12
Rutas de catálogos	12
Rutas de pasajeros	12
Rutas de reservas	12
Rutas de boletos	12
Rutas de equipajes	12
Rutas de reclamos	13
Rutas de vuelos	13

Rutas de pilotos	13
Rutas de asignación de pilotos.....	13
18. Reglas de negocio implementadas	13
Regla 1: Autenticación obligatoria	13
Regla 2: No duplicar reclamos activos por equipaje	13
Regla 3: Estado inicial automático	13
Regla 4: Seguimiento automático.....	14
Regla 5: Reclamo cerrado no cambia de estado	14
Regla 6: Soft delete	14
Regla 7: No repetir piloto en el mismo vuelo	14
Regla 8: No duplicar rol de tripulación	14
19. Soft delete.....	14
20. Pruebas con Thunder Client	14
21. Errores comunes y soluciones	15
Error: Target class does not exist	15
Error: Unknown column	16
Error: Unauthenticated.....	16
Error: CORS.....	16
Error: Table already exists	16
22. Conclusión técnica.....	17

MANUAL TECNICO SISTEMA AEROLINEA RECLAMOS

1. Introducción

El presente manual técnico describe la estructura, instalación, configuración y funcionamiento interno del Sistema de Gestión de Reclamos de Equipaje BOA. El sistema fue desarrollado con una arquitectura separada entre backend y frontend.

El backend fue implementado con Laravel 9, funcionando como una API REST encargada de gestionar la lógica de negocio, autenticación, validaciones, conexión a base de datos y endpoints del sistema. El frontend fue desarrollado de forma independiente utilizando HTML, CSS y JavaScript puro, consumiendo los servicios proporcionados por el backend mediante peticiones HTTP.

El sistema permite registrar pasajeros, reservas, boletos, equipajes, reclamos, seguimiento de reclamos, vuelos, pilotos y asignaciones de pilotos a vuelos. Además, incorpora autenticación mediante tokens, reglas de negocio, soft delete y una estructura de base de datos normalizada para evitar redundancia de información.

2. Objetivo del sistema

El objetivo principal del sistema es permitir la gestión ordenada de reclamos de equipaje dentro de una aerolínea, centralizando la información de pasajeros, vuelos, boletos, equipajes y estados de reclamos.

El sistema busca facilitar:

- Registro de pasajeros.
- Registro de reservas.
- Emisión de boletos.
- Registro de equipajes.
- Creación de reclamos por pérdida, daño o demora.
- Cambio de estados del reclamo.
- Registro automático del seguimiento del reclamo.
- Gestión de vuelos.
- Gestión de pilotos.
- Asignación de pilotos a vuelos.

- Consulta de información desde una interfaz web separada.
- Pruebas de endpoints mediante Thunder Client.

3. Arquitectura del proyecto

El proyecto trabaja con una arquitectura cliente-servidor.

Por un lado, el backend Laravel se encarga de la API, la base de datos y las reglas de negocio. Por otro lado, el frontend independiente se encarga de mostrar la interfaz visual al usuario y consumir los endpoints mediante JavaScript.

La estructura general es la siguiente:

Frontend HTML/CSS/JavaScript
↓ consume API mediante fetch()
Backend Laravel 9 API REST
↓ usa Eloquent y validaciones
Base de datos MySQL

La separación entre frontend y backend permite que el sistema sea más ordenado, mantenible y fácil de probar.

4. Tecnologías utilizadas

Backend

- PHP 8
- Laravel 9
- Laravel Sanctum
- MySQL
- Composer
- Eloquent ORM
- Migraciones
- Seeders
- API REST
- Soft Deletes

Frontend

- HTML5
- CSS3

- JavaScript puro
- Fetch API
- Live Server

Herramientas de desarrollo

- Visual Studio Code
- XAMPP
- phpMyAdmin
- Thunder Client
- Navegador web

5. Requisitos para ejecutar el proyecto

Para ejecutar correctamente el sistema se necesita tener instalado:

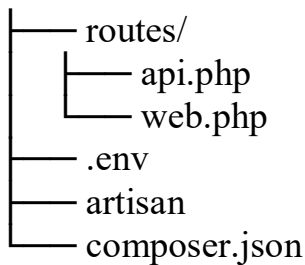
- PHP 8 o superior
- Composer
- XAMPP o servidor MySQL
- Visual Studio Code
- Thunder Client
- Live Server
- Navegador web

También se debe contar con una base de datos creada en phpMyAdmin.

6. Estructura del backend

La estructura principal del backend Laravel es:

```
aerolinea-boa/
├── app/
│   ├── Http/
│   │   ├── Controllers/
│   │   │   └── API/
│   │   └── Resources/
│   └── Models/
├── config/
├── database/
│   ├── migrations/
│   └── seeders/
```



La carpeta app/Models contiene los modelos de las tablas.

La carpeta app/Http/Controllers/API contiene los controladores de la API.

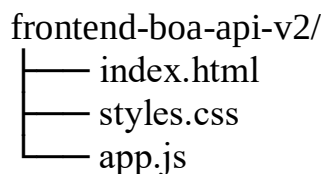
La carpeta database/migrations contiene la estructura de la base de datos.

La carpeta database/seeder contiene los datos iniciales.

La carpeta routes/api.php contiene los endpoints del sistema.

7. Estructura del frontend

La estructura del frontend es independiente del backend:



El archivo index.html contiene la estructura visual.

El archivo styles.css contiene los estilos de la interfaz.

El archivo app.js contiene la lógica de conexión con la API.

El frontend se ejecuta con Live Server y consume el backend mediante esta URL base:

```
const API_BASE = 'http://127.0.0.1:8000/api/v1';
```

8. Configuración del archivo .env

El archivo .env se encuentra en la raíz del proyecto Laravel. En este archivo se configura la conexión a la base de datos.

Ejemplo:

```
APP_NAME=BOA_API
APP_ENV=local
APP_DEBUG=true
APP_URL=http://127.0.0.1:8000
```

```
DB_CONNECTION=mysql
```

```
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=sistema_boa  
DB_USERNAME=root  
DB_PASSWORD=
```

El valor de DB_DATABASE debe coincidir con el nombre de la base de datos creada en phpMyAdmin.

9. Instalación del backend

Primero se debe abrir la carpeta del backend Laravel en Visual Studio Code.

Luego se abre una terminal en la raíz del proyecto y se ejecutan los siguientes comandos:

```
composer install
```

Este comando instala las dependencias del proyecto.

Luego se debe limpiar la configuración de Laravel:

```
php artisan optimize:clear
```

Después se genera o actualiza el autoload de Composer:

```
composer dump-autoload
```

Si Laravel Sanctum no está instalado, se debe ejecutar:

```
composer require laravel/sanctum
```

Luego se ejecutan las migraciones y seeders:

```
php artisan migrate:fresh --seed
```

Finalmente se levanta el servidor Laravel:

```
php artisan serve
```

El backend quedará disponible en:

```
http://127.0.0.1:8000
```

10. Instalación del frontend

El frontend debe estar separado del proyecto Laravel.

Se abre la carpeta:

frontend-boa-api-v2/

Luego se abre el archivo index.html con Live Server.

La interfaz quedará disponible en una URL similar a:

<http://127.0.0.1:5500/index.html>

Para que el frontend funcione correctamente, el backend Laravel debe estar ejecutándose en:

<http://127.0.0.1:8000>

11. Modelo relacional normalizado

El sistema utiliza una base de datos normalizada para evitar redundancia de datos.

Las tablas principales son:

roles
usuarios
países
ciudades
aeropuertos
aviones
pilotos
vuelos
asignacion_pilotos
pasajeros
reservas
boletos
equipajes
tipos_reclamo
estados_reclamo
reclamos
seguimiento_reclamos
personal_access_tokens

Relaciones principales

Un país puede tener muchas ciudades.

países 1 — N ciudades

Una ciudad puede tener muchos aeropuertos.

ciudades 1 — N aeropuertos

Un aeropuerto puede ser origen o destino de muchos vuelos.

aeropuertos 1 — N vuelos

Un avión puede estar asociado a muchos vuelos.

aviones 1 — N vuelos

Un vuelo puede tener varios pilotos y un piloto puede participar en varios vuelos mediante la tabla intermedia asignacion_pilotos.

vuelos N — N pilotos

Un pasajero puede tener muchas reservas.

pasajeros 1 — N reservas

Una reserva pertenece a un vuelo.

vuelos 1 — N reservas

Una reserva puede tener un boleto.

reservas 1 — N boletos

Un boleto puede tener equipajes.

boletos 1 — N equipajes

Un equipaje puede generar un reclamo.

equipajes 1 — N reclamos

Un reclamo tiene un tipo de reclamo.

tipos_reclamo 1 — N reclamos

Un reclamo tiene un estado actual.

estados_reclamo 1 — N reclamos

Un reclamo puede tener varios seguimientos.

reclamos 1 — N seguimiento_reclamos

12. Normalización y reducción de redundancia

Inicialmente se podía tener el campo pais dentro de la tabla ciudades, lo cual generaba repetición de datos como:

La Paz Bolivia
Santa Cruz Bolivia
Cochabamba Bolivia

Para evitar esta redundancia, se creó la tabla paises y la tabla ciudades ahora se relaciona mediante id_pais.

Antes:

ciudades

- id
- nombre_ciudad
- pais

Ahora:

paises

- id
- nombre_pais
- codigo_iso

ciudades

- id
- nombre_ciudad
- id_pais

También se normalizó la tabla aeropuertos, evitando guardar ciudad y país como texto.

Antes:

aeropuertos

- nombre
- ciudad
- pais

Ahora:

aeropuertos

- nombre

- codigo_iata
- id_ciudad

El país se obtiene mediante la relación:

aeropuerto → ciudad → país

De la misma forma, los tipos de reclamo ya no se guardan como texto repetido en la tabla reclamos. Ahora se utiliza la tabla tipos_reclamo.

Antes:

reclamos
- tipo_reclamo

Ahora:

tipos_reclamo
- id
- codigo
- nombre_tipo

reclamos
- id_tipo_reclamo

Esto evita errores como:

PERDIDA

Perdida

Pérdida

perdida

13. Modelos principales

Los modelos principales del sistema se encuentran en:

app/Models/

Algunos modelos importantes son:

- Usuario.php
- Rol.php
- Pais.php
- Ciudad.php
- Aeropuerto.php

- Avion.php
- Piloto.php
- Vuelo.php
- AsignacionPiloto.php
- Pasajero.php
- Reserva.php
- Boleto.php
- Equipaje.php
- TipoReclamo.php
- EstadoReclamo.php
- Reclamo.php
- SeguimientoReclamo.php

Cada modelo representa una tabla de la base de datos y define las relaciones correspondientes.

Ejemplo de relación:

```
public function ciudad()
{
    return $this->belongsTo(Ciudad::class, 'id_ciudad');
}
```

Esta relación indica que un aeropuerto pertenece a una ciudad.

14. Controladores API

Los controladores se encuentran en:

app/Http/Controllers/API/

Los controladores principales son:

- AuthController.php
- CatalogoController.php
- PasajeroController.php
- ReservaController.php
- BoletoController.php
- EquipajeController.php
- ReclamoController.php

- VueloController.php
- PilotoController.php
- AsignacionPilotoController.php

Cada controlador administra las operaciones de una entidad.

Por ejemplo, PasajeroController permite listar, crear, mostrar, actualizar y eliminar pasajeros.

ReclamoController administra la creación de reclamos, cambio de estados, consulta de seguimientos y eliminación lógica.

15. Autenticación

El sistema utiliza Laravel Sanctum para la autenticación mediante tokens.

El usuario inicia sesión enviando sus credenciales. Si las credenciales son correctas, Laravel genera un token de acceso.

Este token debe enviarse en los headers de las peticiones protegidas:

Authorization: Bearer TOKEN_GENERADO

La autenticación permite proteger los endpoints para que solo usuarios registrados puedan consumir la API.

16. Usuarios de prueba

Los usuarios iniciales se crean desde el seeder.

Usuario administrador:

Usuario: admin

Contraseña: admin123

Usuario operador:

Usuario: operador

Contraseña: operador123

Estos usuarios permiten realizar pruebas en Thunder Client y en el frontend

17. Rutas principales de la API

Las rutas se encuentran en:

routes/api.php

La base de la API es:

<http://127.0.0.1:8000/api/v1>

Rutas de autenticación

POST /auth/login

POST /auth/register

GET /auth/me

POST /auth/logout

Rutas de catálogos

GET /catalogos

Rutas de pasajeros

GET /pasajeros

POST /pasajeros

GET /pasajeros/{id}

PUT /pasajeros/{id}

DELETE /pasajeros/{id}

Rutas de reservas

GET /reservas

POST /reservas

GET /reservas/{id}

PUT /reservas/{id}

DELETE /reservas/{id}

Rutas de boletos

GET /boletos

POST /boletos

GET /boletos/{id}

PUT /boletos/{id}

DELETE /boletos/{id}

Rutas de equipajes

GET /equipajes

POST /equipajes

GET /equipajes/{id}

PUT /equipajes/{id}

DELETE /equipajes/{id}

Rutas de reclamos

GET /reclamos
POST /reclamos
GET /reclamos/{id}
PATCH /reclamos/{id}/estado
GET /reclamos/{id}/seguimientos
DELETE /reclamos/{id}

Rutas de vuelos

GET /vuelos
POST /vuelos
GET /vuelos/{id}
PUT /vuelos/{id}
DELETE /vuelos/{id}

Rutas de pilotos

GET /pilotos
POST /pilotos
GET /pilotos/{id}
PUT /pilotos/{id}
DELETE /pilotos/{id}

Rutas de asignación de pilotos

GET /asignacion-pilotos
POST /asignacion-pilotos
DELETE /asignacion-pilotos/{id}

18. Reglas de negocio implementadas

El sistema contempla varias reglas de negocio importantes.

Regla 1: Autenticación obligatoria

El usuario debe iniciar sesión para acceder a las rutas protegidas.

Regla 2: No duplicar reclamos activos por equipaje

Un mismo equipaje no debe tener dos reclamos activos al mismo tiempo.

Regla 3: Estado inicial automático

Cuando se crea un reclamo, el sistema asigna automáticamente el estado inicial PENDIENTE.

Regla 4: Seguimiento automático

Cuando se crea o actualiza un reclamo, el sistema registra un seguimiento en la tabla seguimiento_reclamos.

Regla 5: Reclamo cerrado no cambia de estado

Si un reclamo está cerrado, no debe volver a cambiar de estado.

Regla 6: Soft delete

Cuando se elimina un registro operativo, no se borra físicamente, sino que se llena la columna deleted_at.

Regla 7: No repetir piloto en el mismo vuelo

Un piloto no puede ser asignado dos veces al mismo vuelo.

Regla 8: No duplicar rol de tripulación

Un vuelo no debe tener dos capitanes ni dos copilotos asignados al mismo tiempo.

19. Soft delete

El sistema utiliza eliminación lógica o soft delete. Esto significa que los registros no se eliminan físicamente de la base de datos.

Cuando se elimina un registro, Laravel llena la columna:

deleted_at

Esto permite mantener el historial y recuperar información si fuera necesario.

Ejemplo:

```
DELETE /api/v1/reclamos/1
```

El registro ya no aparece en las consultas normales, pero sigue existiendo en la base de datos con el campo deleted_at lleno.

20. Pruebas con Thunder Client

El flujo recomendado para probar el sistema es:

1. Login
2. Consultar usuario autenticado

3. Consultar catálogos
4. Crear pasajero
5. Crear reserva
6. Crear boleto
7. Crear equipaje
8. Crear reclamo
9. Listar reclamos
10. Buscar reclamo
11. Ver detalle de reclamo
12. Cambiar estado del reclamo
13. Ver seguimiento
14. Probar reglas de negocio
15. Eliminar con soft delete
16. Probar pilotos
17. Probar asignaciones
18. Logout

Los headers principales son:

Accept: application/json

Content-Type: application/json

Authorization: Bearer TOKEN_GENERADO

21. Errores comunes y soluciones

Error: Target class does not exist

Este error ocurre cuando un controlador está fuera de la carpeta correcta o el namespace no coincide.

Solución:

Verificar que los controladores estén en:

app/Http/Controllers/API/

Y que tengan este namespace:

namespace App\Http\Controllers\API;

Luego ejecutar:

```
php artisan optimize:clear  
composer dump-autoload
```

Error: Unknown column

Este error aparece cuando Laravel consulta una columna que no existe en la base de datos.

Ejemplo:

Unknown column 'nombre_aeropuerto'

Solución:

Revisar que el código use el nombre real de la columna. En este caso debe ser: nombre

no:

nombre_aeropuerto

Error: Unauthenticated

Este error aparece cuando se intenta consumir una ruta protegida sin token.

Solución:

Iniciar sesión y enviar el token en el header:

Authorization: Bearer TOKEN_GENERADO

Error: CORS

Este error aparece cuando el frontend no puede comunicarse con el backend.

Solución:

Revisar config/cors.php y ejecutar:

php artisan optimize:clear

Error: Table already exists

Este error aparece cuando existen migraciones duplicadas que intentan crear la misma tabla.

Solución:

Eliminar migraciones antiguas duplicadas y ejecutar:

php artisan migrate:fresh --seed

22. Conclusión técnica

El sistema fue desarrollado con una arquitectura separada entre backend y frontend. El backend utiliza Laravel 9 como API REST, con autenticación mediante tokens, migraciones, seeders, modelos Eloquent, validaciones y reglas de negocio. El frontend consume la API mediante JavaScript puro y permite al usuario interactuar con el sistema de manera visual.

La base de datos fue normalizada para evitar redundancia de información, separando entidades como países, ciudades, aeropuertos, tipos de reclamo y estados de reclamo. Además, se implementó soft delete para conservar trazabilidad de los registros eliminados.

El sistema permite realizar un flujo completo desde el registro de pasajero hasta la gestión del reclamo y su seguimiento.